Pseudocode:

Camera Part:

- Used OpenCV to detect ArUco tag centers (x,y) in pixels and their tag ID
- Used our math transform to convert the camera (x,y) into the table frame (x,y,z) coordinate for the robot
- **Math Transform:**
    - For each marker index:
        - marker_pixel is a 3D vector of the x component, y component, and 1
        - projected_point is the perspective matrix times the marker_pixel
        - projected_point x and y coordinates are divided by the third coordinate, to normalize the vectors
        - Add projected_point vectors to the ith term in the marker_centers_in_table_frame matrix
        - end
    - Return marker_centers_in_table_frame

- RETURN list of marker centers in the table frame

UR3e Controller

This class talks to the UR3e robot arm and controls its movements.

When we create the controller:

- Set up a "home" position where the arm starts
- Remember where the arm currently is
- Set default speed and acceleration
- Connect to the robot's communication channels (publishers and subscribers)

When the robot tells us its current joint positions:

- Update our memory of where the arm is right now
- Mark that we've received at least one position update

When the robot tells us about the gripper vacuum sensor:

- Remember whether the gripper is currently sucking or not

move_arm()

- Create a command message with: the target joint angles we want, the desired velocity and acceleration, and whether to keep the gripper on or off (don't change it)
- Send the command to the robot
- Keep checking if the arm has reached the target
- If it takes too long, give up and warn the user

gripper_control()

- Figure out what we want the gripper to do (suck or release)
- Create a command that keeps the arm in place but changes the gripper
- Send the command
- Wait a bit for the gripper to respond

at_goal()

- Look at each joint angle
- If all joints are within a small tolerance of the target, we're good
- Otherwise, we need to keep waiting

—----------

Block Mover

This class coordinates the whole pick-and-place operation. It uses the robot controller and the ArUco tracker to find blocks and move them.

To move a single block from one place to another:

This is the main pick-and-place sequence:

1. Move the arm to hover above where the block is (at a safe height)

2. Lower the arm down to the block's height

3. Turn on the vacuum gripper to grab the block

4. Lift the block up to a safe height

5. Move the arm (with the block) to hover above where we want to drop it

6. Lower the arm down to the drop location

7. Turn off the vacuum gripper to release the block

The block is now in its new location!

To process all the blocks we can see:
This is the main function:

1. First, wait for the camera system to detect at least one ArUco marker
   - Keep checking every so often
   - If we wait more than 10 seconds without detecting an ArUco marker, timeout

2. Once we have detections, get the list of marker IDs and their positions
   - The positions are already converted to table coordinates (in millimeters)

3. If we didn't find any markers, stop and report the problem

4. Move the robot arm to its home position (so we start from a known place)

5. Now we'll stack the blocks one on top of another:

   For the first block:
   - Convert its position from millimeters to meters
   - Figure out where on the stack it should go (the bottom)
   - Move it there using the pick-and-place sequence
   - Remember that the stack now needs to be one block taller

   For the second block:
   - Convert its position to meters
   - Figure out where on the stack it should go (on top of the first)
   - Move it there
   - Remember that the stack now needs to be two blocks taller

   For the third block:
   - Convert its position to meters
   - Figure out where on the stack it should go (on top of the second)
   - Move it there
   - The stack is now complete

Main Pipeline Instructions

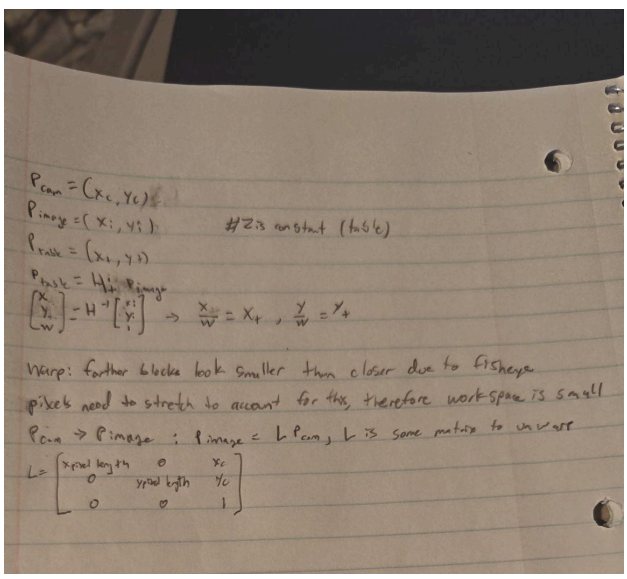
1. Initialize the ROS2 communication system
2. Set up the robot system components:-
    - Create a robot controller
    - Create an ArUco tracker
    - Wait for the robot to report its current position
    - Define three destination spots for blocks (yellow, red, and blue)
    - Select one destination spot for stacking
    - Create a block mover that interfaces with both the robot controller and tracker
3. Start the ArUco tracker's continuous monitoring loop:
    - Watch the camera feed for ArUco markers
    - Detect markers in the image
    - Calculate marker positions in real-world coordinates
4. Begin the block mover's processing sequence:
    - Request the list of observed blocks from the tracker
    - Receive marker IDs and world-frame positions for each block
5. For each detected block, execute the movement sequence:
    - Determine the block's current position
    - Decide the target location (either outside the workspace or in a stack)
    - Command the robot controller to move the block using move_arm()
6. The robot controller breaks down each block move into sequential steps in move_arm():

- Move to the approach position above the block
- Lower the end effector to the grab position
- Activate the gripper
- Lift up with the block secured
- Move to the drop location
- Lower to place position
- Deactivate the gripper to release the block

7. Move the robot using kinematics functions to calculate joint angles and trajectories
8. Repeat steps 5-7 until all blocks are stacked

Move blocks out of workspace: https://www.youtube.com/shorts/E2p7oBDGI3E
Stack blocks: https://www.youtube.com/shorts/e-HkDXCaxnE?feature=share

Math:



$P_{cam} = (x_c, y_c)$

$P_{image} = (x_i, y_i)$ #Z is constant (table)

$P_{task} = (x_t, y_t)$

$P_{task} = H_{ti} P_{image}$

$\begin{bmatrix} x_t \\ y_t \\ w \end{bmatrix} = H^{-1} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} \rightarrow \frac{x_t}{w} = x_t, \frac{y}{w} = y_t$

Warp: farther blocks look smaller than closer due to fisheye
pixels need to stretch to account for this, therefore workspace is small

$P_{cam} \rightarrow P_{image}$: $P_{image} = L P_{cam}$, L is some matrix to unwarp

$L = \begin{bmatrix} \text{x pixel length} & 0 & x_c \\ 0 & \text{y pixel length} & y_c \\ 0 & 0 & 1 \end{bmatrix}$